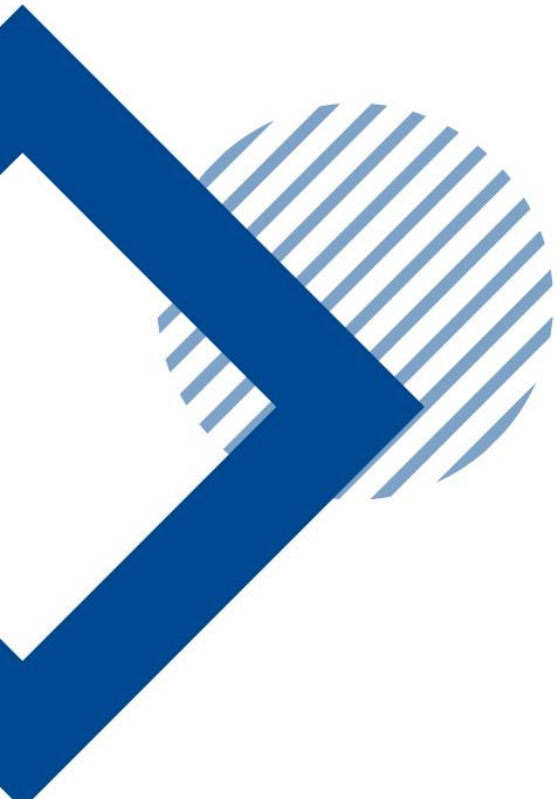
Безопасность

Интернет программирование





2022

Основные понятия

Аутентификация – определение пользователя по его логину/паролю (кто это?)

Авторизация – определение прав доступа пользователя (может ли пользователь выполнять действие?)

Виды аутентификации

1. Аутентификация по паролю.
- HTTP-аутентификация;
- Формы/тело запроса;
- Заголовок запроса.
2. Аутентификация по сертификатам.
3. Аутентификация по одноразовым паролям.
4. Аутентификация по ключам доступа.
5. Аутентификация по токенам.
- SWT;
- JWT;
- SAML;
- OAuth;
- OpenID.

Аутентификация по паролю

Пользователь должен предоставить имя (username/login) и пароль (password) для успешной идентификации и аутентификации в системе.

Пара username/password задается пользователем при его регистрации в системе, при этом в качестве username может выступать адрес электронной почты пользователя.

HTTP authentication – 1

Сервер, при обращении анонимного клиента к защищенному ресурсу, отсылает HTTP статус “401 Unauthorized” и добавляет заголовок “WWW-Authenticate” с указанием схемы и параметров аутентификации.

Браузер, при получении такого ответа, автоматически показывает диалог ввода username и password. Пользователь вводит детали своей учетной записи.

Во всех последующих запросах к этому веб-сайту браузер автоматически добавляет HTTP заголовок “Authorization”, в котором передаются данные пользователя для аутентификации сервером.

Сервер идентифицирует пользователя по данным из этого заголовка. Решение о предоставлении доступа (авторизация) производится отдельно на основании роли пользователя, ACL или других данных учетной записи.

HTTP authentication – 2

Существует несколько схем аутентификации, отличающихся по уровню безопасности:

- Basic — наиболее простая схема, при которой username и password пользователя передаются в заголовке Authorization в незашифрованном виде (base64-encoded). Однако при использовании HTTPS (HTTP over SSL) протокола, является относительно безопасной.
- Digest — challenge-response-схема, при которой сервер посылает уникальное значение nonce, а браузер передает MD5 хэш пароля пользователя, вычисленный с использованием указанного nonce.
- NTLM (известная как Windows authentication) — также основана на challenge-response подходе, при котором пароль не передается в чистом виде. Эта схема не является стандартом HTTP, но поддерживается большинством браузеров и веб-серверов.
- Negotiate — еще одна схема из семейства Windows authentication, которая позволяет клиенту выбрать между NTLM и Kerberos аутентификацией. Kerberos — более безопасный протокол, основанный на принципе Single Sign-On. Однако он может функционировать, только если и клиент, и сервер находятся в зоне intranet и являются частью домена Windows.

Стоит отметить, что при использовании HTTP-аутентификации у пользователя нет стандартной возможности выйти из веб-приложения, кроме как закрыть все окна браузера.

Forms authentication

В веб-приложение включается HTML-форма, в которую пользователь должен ввести свои username/password и отправить их на сервер через HTTP POST запрос для аутентификации.

В случае успеха веб-приложение создает сессионный токен (session token), который обычно помещается в куки браузера (browser cookies).

При последующих веб-запросах session token автоматически передается на сервер и позволяет приложению получить информацию о текущем пользователе для авторизации запроса.

Приложение может создать session token двумя способами:

1. Как идентификатор сессии пользователя, которая хранится в памяти сервера или в базе данных. Сессия должна содержать всю необходимую информацию о пользователе для возможности авторизации его запросов.
2. Как зашифрованный и/или подписанный объект, содержащий данные о пользователе, а также период действия. Этот подход позволяет реализовать stateless-архитектуру сервера, однако требует механизма обновления сессионного токена по истечении срока действия. Несколько стандартных форматов таких токенов рассматриваются в секции «Аутентификация по токенам».

Аутентификация по сертификатам – 1

Сертификат (подписанный certificate authority (CA)) представляет собой набор атрибутов, идентифицирующих владельца.

CA выступает в роли посредника, который гарантирует подлинность сертификатов. Также сертификат криптографически связан с закрытым ключом, которых хранится у владельца сертификата и позволяет однозначно подтвердить факт владения сертификатом.

На стороне клиента сертификат вместе с закрытым ключом могут храниться в операционной системе, в браузере, в файле, на отдельном физическом устройстве (smart card, USB token). Обычно закрытый ключ дополнительно защищен паролем или PIN-кодом.

В веб-приложениях традиционно используют сертификаты стандарта X.509. Аутентификация с помощью X.509-сертификата происходит в момент соединения с сервером и является частью протокола SSL/TLS. Этот механизм также хорошо поддерживается браузерами, которые позволяют пользователю выбрать и применить сертификат, если веб-сайт допускает такой способ аутентификации.

Аутентификация по сертификатам – 2

Во время аутентификации сервер выполняет проверку сертификата на основании следующих правил:

1. Сертификат должен быть подписан доверенным certification authority (проверка цепочки сертификатов).
2. Сертификат должен быть действительным на текущую дату (проверка срока действия).
3. Сертификат не должен быть отозван соответствующим CA (проверка списков исключения).

После успешной аутентификации веб-приложение может выполнить авторизацию запроса на основании таких данных сертификата, как subject (имя владельца), issuer (эмитент), serial number (серийный номер сертификата) или thumbprint (отпечаток открытого ключа сертификата).

Использование сертификатов для аутентификации — куда более надежный способ, чем аутентификация посредством паролей. Это достигается созданием в процессе аутентификации цифровой подписи, наличие которой доказывает факт применения закрытого ключа в конкретной ситуации.

Аутентификация по одноразовым паролям

Аутентификация по одноразовым паролям обычно применяется дополнительно к аутентификации по паролям для реализации two-factor authentication (2FA). В этой концепции пользователю необходимо предоставить данные двух типов для входа в систему: что-то, что он знает (например, пароль), и что-то, чем он владеет (например, устройство для генерации одноразовых паролей). Наличие двух факторов позволяет в значительной степени увеличить уровень безопасности, что м. б. востребовано для определенных видов веб-приложений.

Другой популярный сценарий использования одноразовых паролей — дополнительная аутентификация пользователя во время выполнения важных действий: перевод денег, изменение настроек и т. п.

Аутентификация по ключам доступа

Этот способ чаще всего используется для аутентификации устройств, сервисов или других приложений при обращении к веб-сервисам. Здесь в качестве секрета применяются ключи доступа (access key, API key) — длинные уникальные строки, содержащие произвольный набор символов, по сути заменяющие собой комбинацию username/password.

В большинстве случаев, сервер генерирует ключи доступа по запросу пользователей, которые далее сохраняют эти ключи в клиентских приложениях. При создании ключа также возможно ограничить срок действия и уровень доступа, который получит клиентское приложение при аутентификации с помощью этого ключа.

Аутентификация по токенам – 1

Такой способ аутентификации чаще всего применяется при построении распределенных систем Single Sign-On (SSO), где одно приложение (service provider или relying party) делегирует функцию аутентификации пользователей другому приложению (identity provider или authentication service). Типичный пример этого способа — вход в приложение через учетную запись в социальных сетях. Здесь социальные сети являются сервисами аутентификации, а приложение доверяет функцию аутентификации пользователей социальным сетям.

Реализация этого способа заключается в том, что identity provider (IP) предоставляет достоверные сведения о пользователе в виде токена, а service provider (SP) приложение использует этот токен для идентификации, аутентификации и авторизации пользователя.

Аутентификация по токенам – 2

На общем уровне, весь процесс выглядит следующим образом:

1. Клиент идентифицируется в identity provider одним из способов, специфичным для него (пароль, ключ доступа, сертификат, Kerberos, итд.).
2. Клиент просит identity provider предоставить ему токен для конкретного SP-приложения. Identity provider генерирует токен и отправляет его клиенту.
3. Клиент идентифицируется в SP-приложении при помощи этого токена.

Существует несколько стандартов, в точности определяющих протокол взаимодействия между клиентами (активными и пассивными) и IP/SP-приложениями и формат поддерживаемых токенов.

Среди наиболее популярных стандартов — OAuth, OpenID Connect, SAML, и WS-Federation.

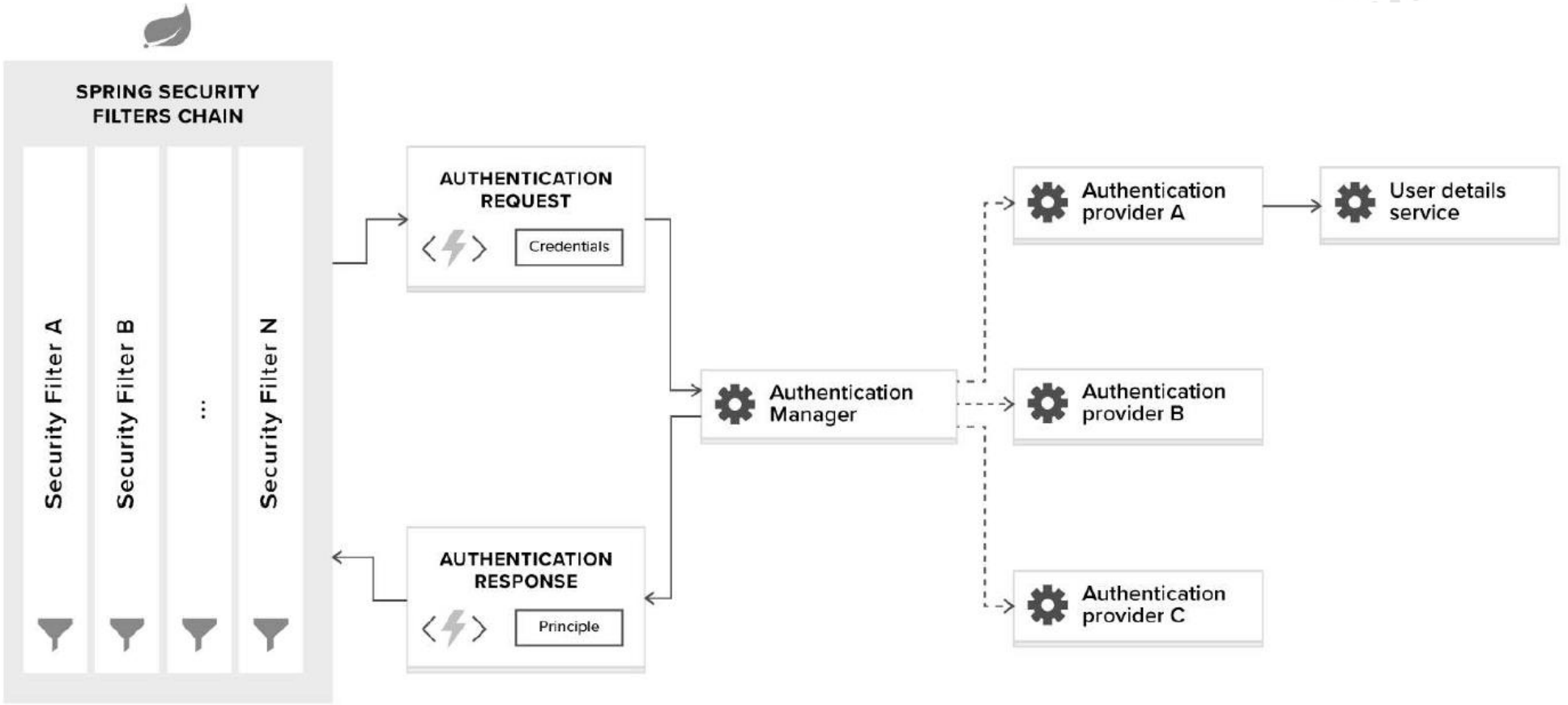
Spring Security – 1

User Details (Principle) – ссылка на текущего пользователя, прошедшего аутентификацию.

Granted authority – ссылка на права доступа текущего пользователя.

Role/Roles – ссылка на коллекцию прав доступа текущего пользователя.

Spring Security – 2



Spring Security – 3

AuthenticationManager определяет провайдер для обработки запроса по его типу.

AuthenticationProvider реализует определенный тип аутентификации.

UserDetailsService интерфейс для реализации функции получения данных о пользователе.

Spring Security – Зависимости

implementation 'org.springframework.boot:spring-boot-starter-security‘
implementation 'org.thymeleaf.extras:thymeleaf-extras-springsecurity5’

```
<!DOCTYPE html>
<html lang="ru„
     xmlns:th=http://www.thymeleaf.org
     xmlns:layout=http://www.ultraq.net.nz/thymeleaf/layout
     xmlns:sec="http://www.thymeleaf.org/thymeleaf-extras-springsecurity5">

<body>
<th:block sec:authorize="isAuthenticated()">
    <th:block sec:authorize="hasRole('ADMIN’)”>
    </th:block>
    <th:block sec:authorize="hasRole('USER’)”>
    </th:block>
    <span th:text="${#authentication.name}"></span>
</th:block>

</body>

</html>
```

Аутентификация в Spring Security – 1

```
public class UserPrincipal implements UserDetails {
    private final long id;
    private final String username;
    private final String password;
    private final Set<? extends GrantedAuthority> roles;
    private final boolean active;

    public UserPrincipal(UserEntity user) {
        ...
    }

    // getter & setters
}

@Service
public class UserService implements UserDetailsService {
    @Override
    @Transactional(readOnly = true)
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
        final UserEntity existsUser = getByLogin(username);
        return new UserPrincipal(existsUser);
    }
}
```

Аутентификация в Spring Security – 2

```
@Configuration
@EnableWebSecurity
public class SecurityConfiguration {
  ...

   @Bean
   DaoAuthenticationProvider authenticationProvider(UserDetailsService userDetailsService) {
      final DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
      authProvider.setUserDetailsService(userDetailsService);
      authProvider.setPasswordEncoder(passwordEncoder());
      return authProvider;
   }

   @Bean
   PasswordEncoder passwordEncoder() {
      return new BCryptPasswordEncoder();
   }
}
```

Аутентификация в Spring Security – 3

```
@Configuration
@EnableWebSecurity
public class SecurityConfiguration {
   @Bean
   SecurityFilterChain filterChain(HttpSecurity httpSecurity) throws Exception {
      httpSecurity.formLogin(formLogin -> formLogin
            .loginPage(Constants.LOGIN_URL));

      httpSecurity.rememberMe(rememberMe -> rememberMe.key("uniqueAndSecret"));

      httpSecurity.logout(logout -> logout
            .deleteCookies("JSESSIONID"));
      return httpSecurity.build();
   }
}

@Configuration
public class WebConfiguration implements WebMvcConfigurer {
   @Override
   public void addViewControllers(ViewControllerRegistry registry) {
      registry.addViewController("/login").setViewName("login");
   }
}
```

Аутентификация в Spring Security – 4

```
<form action="#" th:action="@{/login}" method="post">
    <div class="mb-3">
        <label for="username" class="form-label">Имя пользователя</label>
        <input type="text" id="username" name="username" class="form-control" required
          minlength="3" maxlength="20">
    </div>
    <div class="mb-3">
        <label for="password" class="form-label">Пароль</label>
        <input type="password" id="password" name="password" class="form-control" required
           minlength="3" maxlength="20">
    </div>
    <div class="form-check mb-3">
        <input class="form-check-input" type="checkbox" id="remember-me«
          name="remember-me" checked>
        <label class="form-check-label" for="remember-me">Запомнить меня</label>
    </div>
    <div class="mb-3 d-flex flex-row">
        <button class="btn btn-primary me-2 button-fixed-width«
          type="submit">Войти</button>
        <a class="btn btn-secondary button-fixed-width" href="/signup">Регистрация</a>
    </div>
</form>
```

Авторизация в Spring Security – 1

```
public enum UserRole implements GrantedAuthority {
    ADMIN,
    USER;

    private static final String PREFIX = "ROLE_";

    @Override
    public String getAuthority() {
        return PREFIX + this.name();
    }
}

@Entity
@Table(name = "users")
public class UserEntity extends BaseEntity {
    ...
    private UserRole role;
    ...
}
```

Авторизация в Spring Security – 2

Через конфигурацию:

```
@Configuration
@EnableWebSecurity
public class SecurityConfiguration {
    @Bean
    SecurityFilterChain filterChain(HttpSecurity httpSecurity) throws Exception {
        httpSecurity.authorizeHttpRequests(requests -> requests
                .requestMatchers("/css/**", "/webjars/**", "/*.svg")
                .permitAll());

        httpSecurity.authorizeHttpRequests(requests -> requests
                .requestMatchers(Constants.ADMIN_PREFIX + "/**").hasRole(UserRole.ADMIN.name())
                .requestMatchers("/h2-console/**").hasRole(UserRole.ADMIN.name())
                .requestMatchers(UserSignupController.URL).anonymous()
                .requestMatchers(Constants.LOGIN_URL).anonymous()
                .anyRequest().authenticated());
    }
}
```

Авторизация в Spring Security – 3

```
@EnableGlobalMethodSecurity(
        securedEnabled = true,
        jsr250Enabled = true,
        prePostEnabled = true
)
```

securedEnabled – включает использование аннотации @Secured;
jsr250Enabled – включает использование аннотации @RolesAllowed;
prePostEnabled – включает использование аннотаций @PreAuthorise, @PostAuthorise, @PreFilter, @PostFilter.

Через аннотации:

- @PreAuthorise – выполняет выражение на языке SEL для контроля доступа перед выполнением метода;
- @PostAuthorise – выполняет выражение на языке SEL для контроля доступа после выполнением метода (доступ к результату);
- @PreFilter – использует выражение SEL для фильтрации перед выполнением;
- @PostFilter – использует выражение SEL для фильтрации после выполнения;
- @Secured – определяет список ролей пользователей, которые могут использовать метод/методы класса;
- @RolesAllowed – аналогично @Secured (реализовано в рамках JSR 250).

Получение сведений о текущем пользователе

```
@PostMapping("/delete/{id}")
public String deleteOrder(
        RedirectAttributes redirectAttributes,
        @AuthenticationPrincipal UserPrincipal principal) {
    orderService.delete(principal.getId(), id);
    return Constants.REDIRECT_VIEW + "/";
}
```

Имя	Значение	Domain	Path	Expire...
JSESSIONID	A9832AF35C3855F1136AA7C1A68DB091	localh...	/	Сеанс
remember-me	YWRtaW46MTcxNDk4NjM1MzkyOTpTSEEy...	localh...	/	2024-...

Спасибо за внимание!

Литература:

1. Security with Spring. URL: https://www.baeldung.com/security-spring
2. Spring Security. URL: https://docs.spring.io/spring-security/reference/index.html
3. Spring Security with JWT for REST API. URL: https://www.toptal.com/spring/spring-security-tutorial

Ульяновский государственный технический университет

2022